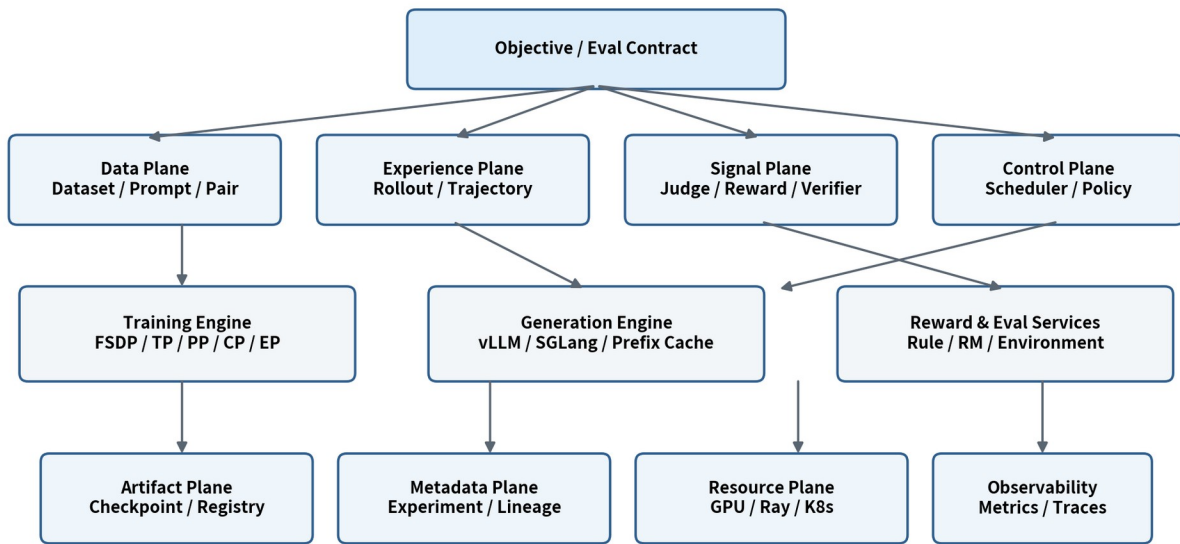


LLM 后训练之 Post-Training Infrastructure

后训练基础设施与平台

深度研究与工程手册



Post-Training Platform = Dataflow + Modelflow + Signalflow + Resourceflow + Governance

核心问题：如何把 Data / Rollout / Reward / Train / Eval 从“若干脚本”升级为可复现、可扩展、可容错、可持续迭代的生产系统？

研究范围：Distributed Training · Rollout Engine · RLHF/RLVR Dataflow · Resource Scheduling · Checkpoint · Registry · Lineage · Observability · Reliability · Cost

执行摘要

后训练基础设施不是“给 SFT/RL 加几台 GPU”。它承担的是五条流的同时编排：数据流（Dataflow）、模型流（Modelflow）、训练信号流（Signalflow）、资源流（Resourceflow）和治理流（Governance）。在 SFT 阶段，系统近似传统分布式训练；进入 RLHF/RLVR 后，系统会同时运行 actor/policy、rollout engine、reward/verifier、reference/value model、environment 与 trainer，且这些组件具有不同的计算强度、显存形态和通信模式。[1][2][3]

第一性结论：Post-Training Platform 的核心优化目标不是 tokens/s，而是“有效学习信号 / GPU-hour”，同时满足 freshness、reproducibility、reliability 与 release safety。

层	核心对象	核心 SLO / 约束
Data Plane	dataset / prompt / pair / trajectory	版本、血缘、去重、可追溯
Experience Plane	rollout / environment interaction	吞吐、p95 长尾、policy freshness
Signal Plane	reward / verifier / judge	延迟、可靠性、版本一致性
Training Plane	SFT/DPO/PPO/GRPO trainer	MFU、显存、collective、稳定性
Control Plane	scheduler / orchestration	资源平衡、背压、重试、staleness
Artifact Plane	checkpoint / registry	原子性、可恢复、reshardability
Eval & Release	eval / regression / gate	独立性、可重复、可回滚

本手册重点从 know-why 和 know-how 两侧回答：为什么 RL 系统比 SFT 平台复杂；为什么 rollout 往往成为瓶颈；为什么 co-located、hybrid、disaggregated、async 各有边界；为什么 checkpoint、lineage 和 eval gate 不是“辅助功能”，而是后训练正确性的组成部分。

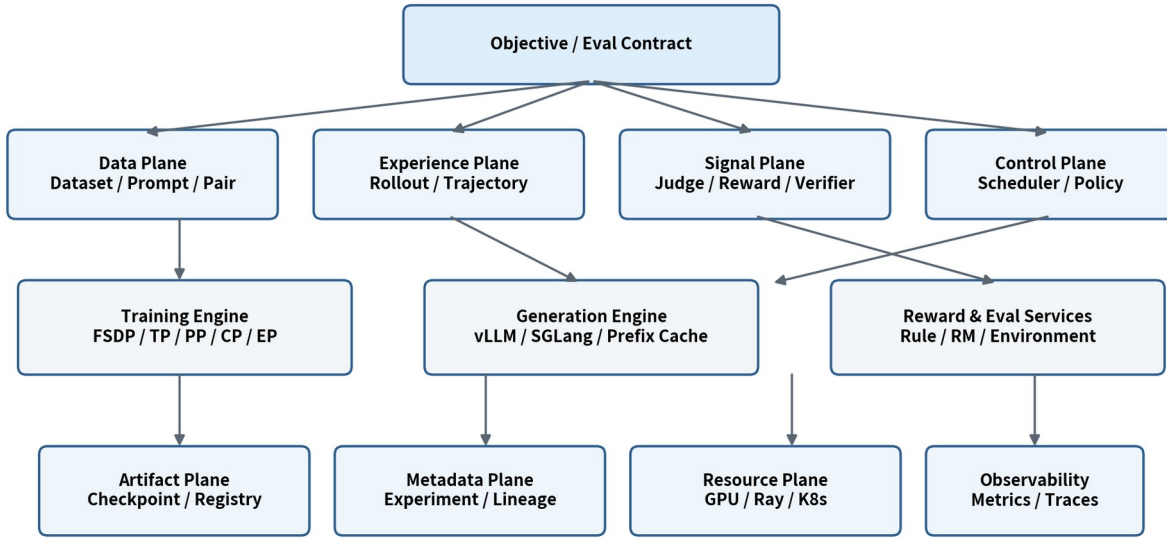
1. 为什么 Post-Training Infrastructure 是独立问题

预训练的主循环可以粗略看成“load batch → forward/backward → optimizer step”。RLHF/RLVR 的主循环则扩展成“sample prompt → rollout → reward/verify → advantage → update → refresh policy → repeat”。HybridFlow 指出，RLHF 将传统 RL dataflow 中的每个节点放大成分布式 LLM 训练/生成程序，每条边也变成多对多的数据与权重交换，因此控制和通信本身成为主要系统问题。[1]

维度	SFT / DPO	PPO / GRPO / RLVR
模型角色	1 个主模型 + 可选 reference	policy + reference + reward/value/verifier 等
数据	大多静态	持续生成的 on/near-policy experience
计算形态	forward/backward 为主	decode-heavy rollout + train-heavy update 混合
资源问题	模型能否放下、训练效率	rollout/train/reward 如何平衡
正确性风险	数据/序列化/optimizer	staleness、reward version、group 完整性、off-policy
系统节拍	step-based	producer-consumer / pipeline-based

OpenRLHF 的系统动机也来自同一问题：传统 co-location 在大模型 RLHF 中会同时受多模型显存压力和资源利用率限制，因此它使用 Ray、vLLM、DeepSpeed 重新组织多模型的调度。[2] DeepSpeed-Chat 则展示了早期把训练与推理统一到一个 RLHF engine 的路线。[3]

2. 平台参考架构：五条流 + 七个平面



Post-Training Platform = Dataflow + Modelflow + Signalflow + Resourceflow + Governance

图 1：Post-Training Platform 的参考分层。

建议不要按“项目目录”划分平台，而按数据/状态语义划分。核心对象如下：

对象	必须携带的身份	为什么重要
Dataset	dataset_id + version + schema	决定训练输入可复现
Prompt	prompt_id + source + task slice	连接 Prompt Factory 与 Eval
Rollout	policy_version + sampling_config + seed	判断 freshness / on-policy
Reward	grader_version + reward_components	避免 reward 漂移不可追踪
Checkpoint	global_step + topology + optimizer state	恢复训练与比较实验
Eval Result	model_version + eval_version + judge_version	防止“同名 benchmark 不同语义”
Release	candidate_id + gates + lineage	实现回滚与审计

平台最危险的反模式：只给 checkpoint 版本号，不给 data/judge/eval/config 版本号。这样“模型 v27”实际上不可复现。

3. Training Engine：从 FSDP 到 5D Parallelism

训练引擎首先解决“一个模型如何高效放到多 GPU 上”。Megatron Core 当前把主要并行维度总结为 DP、TP、PP、CP、EP，并允许组合；总 GPU 数可以近似表示为 $TP \times PP \times CP \times EP \times DP$ 。[4] PyTorch FSDP2 则通过 per-parameter sharding 将 parameters、gradients、optimizer states 分片，并以 DeviceMesh 表达多维设备拓扑。[5][6]

并行方式	切分对象	适用场景	后训练注意点
DP / FSDP	batch / model states	通用训练	DPO/SFT 最常见；GRPO trainer 也常用
TP	层内 tensor	单层太大	通信频繁；rollout 与 train 可能需不同 TP
PP	模型深度	极大模型	pipeline bubble；权重刷新复杂
CP	sequence length	长上下文/长 CoT	长 reasoning 训练可显著降 activation memory
EP	MoE experts	MoE	expert routing + all-to-all 成为瓶颈

Know-how：训练拓扑不是固定配置。Rollout 通常偏向更小 TP、更高 batch concurrency；训练则可能需要 TP/PP/CP/EP 的不同组合。这就是为什么“权重 reshard”成为 RL 平台独有的核心成本。HybridFlow 的 3D-HybridEngine 正是为 training↔generation phase 的 actor resharding 设计。[1]

4. Rollout Engine：后训练最常见的吞吐瓶颈

Reasoning RL 的 rollout 往往是 decode-heavy：输出长度高方差、每个 prompt 需要 N 个 sample、且 batch 必须等待长尾 completion。AReaL 指出同步系统的典型低效来源就是 generation 等待最长输出完成后才能更新模型，导致 GPU idle。[7]

高性能 rollout engine 常用的基础能力包括 continuous batching、KV cache 管理、prefix caching、parallel sampling、speculative decoding、chunked prefill 和分布式推理。vLLM 的官方文档将 prefix caching、distributed serving、parallel sampling、speculative decoding 等列为核心能力。[8]

优化	解决什么	对后训练的特别价值
Continuous batching	减少 decode 空洞	rollout 请求长度差异大
Prefix caching	复用共享前缀 KV	同一 system prompt / few-shot / group prompt
Parallel sampling	同 prompt 多候选	GRPO / Best-of-N
Speculative decoding	降低 token latency	长 CoT / 中低 QPS 场景；需验证总体吞吐
Chunked prefill	控制 prefill 峰值	长 prompt / tool context
Disaggregated prefill/decode	拆不同计算阶段	更灵活资源映射

不要只测单次生成 latency。RL rollout 的关键指标是：tokens/s、samples/s、group completion time、p95/p99 长尾、prefix-cache hit、KV utilization、有效 group/s。

5. Co-located、Hybrid、Disaggregated：三种资源组织方式

Synchronous / co-located



Disaggregated / pipelined

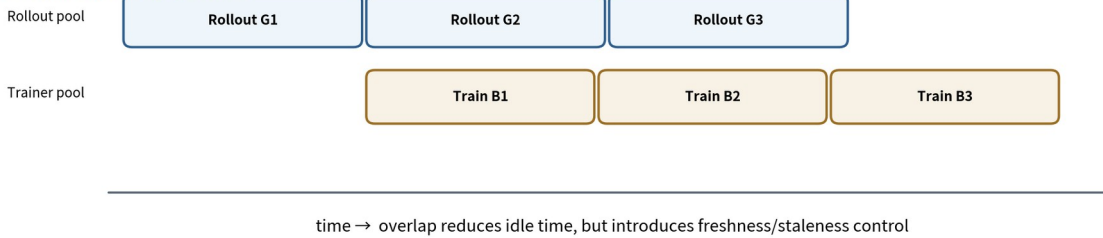


图 2：同步共置与解耦流水线的直观差异。

模式	优点	缺点	适合
Co-located	简单；权重本地；on-policy 容易	角色互相抢显存；训练/生成切换开销	小中规模、研发
Hybrid / Reshard	GPU 可复用；提高利用率	reshard/collective 复杂	中大规模统一资源池
Disaggregated	rollout/train 独立扩缩容	权重同步、staleness、网络要求高	大规模 reasoning RL
Fully Async	最高 overlap 潜力	off-policy/staleness 控制困难	吞吐优先、有稳定化算法

OpenRLHF 偏向用 Ray 将 actor、critic、reward、reference 等角色映射到可独立调度资源。[2] HybridFlow/verl 强调通过 hybrid controller 与 resharding 在灵活性和效率间平衡。[1] AReal 则彻底解耦 generation 和 training，通过 workload balance 与 staleness-aware PPO 控制异步数据陈旧。[7]

6. 同步 vs 异步：核心不是速度，而是 Freshness Contract

异步系统的本质交易是：用更多 overlap 换取更旧的 experience。AReal 报告完全异步架构在相同 GPU 数下获得最高 $2.57\times$ 的训练加速，同时用 workload balance 和 staleness-enhanced PPO 处理陈旧样本。[7] 2026 的 RolloutPipe 则提出另一条路线：不改变 on-policy 语义，而是在同一次固定权重 rollout 内，把已完整 materialize 的 group 提前送入 trainer，从而重叠 rollout 和 training。[9]

Freshness 字段	定义示例	控制方式
policy_version_gen	生成 rollout 时 policy 版本	强制记录
policy_version_train	消费 rollout 时 policy 版本	训练前计算 lag
version_lag	train - gen	hard cap / weighting
wall_clock_age	experience age	TTL
importance ratio	$\pi_{\text{new}}/\pi_{\text{old}}$	clip / off-policy correction

group integrity	同组是否同 policy/配置	GRPO 必须强约束

工程建议：把 staleness 做成数据 schema 的一等字段，而不是训练日志里的临时指标。没有 `policy\_version` 的 trajectory store，无法严谨区分 on-policy、near-policy 和 replay。

7. Resource Scheduler: 后训练是 Producer-Consumer 系统

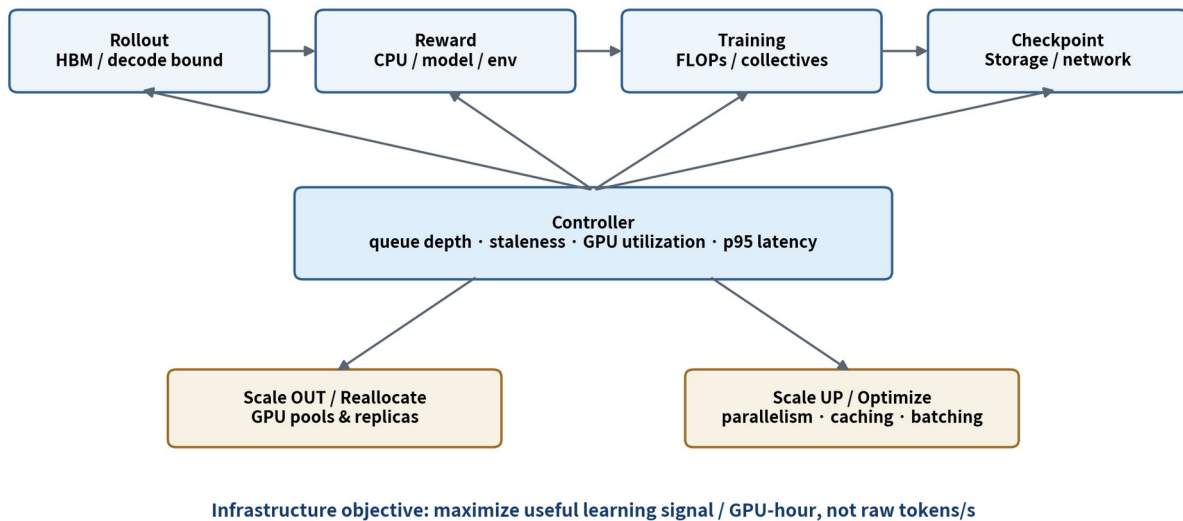


图 3：后训练资源控制器必须围绕瓶颈动态平衡。

Rollout、Reward、Train、Checkpoint 的资源曲线不同：generation 常见 HBM/内存带宽约束，训练更偏 FLOPs/collective，规则 verifier 可能主要吃 CPU，environment 可能受外部 I/O 限制。静态把 50% GPU 给 rollout、50% 给 train 往往并非最优。

Ray Placement Groups 提供 gang scheduling，可原子地预留跨节点资源，并支持 PACK/SPREAD placement，适合多角色 RLHF job 的资源编排。[10] 2026 DynaResize 更进一步，针对 disaggregated RL 在运行时动态把 GPU 在 Rollout 与 Training 角色之间切换，以消除长尾 rollout 导致的 pipeline bubble。[11]

控制信号	过高意味着	动作
rollout queue depth	生成供给不足	增加 rollout replicas/GPU
trainer idle ratio	experience 不够	rollout scale-out / 提前 dispatch
experience backlog	trainer 不够	增加 trainer / 降 rollout
staleness	生产过快或训练过慢	限流 / drop old / increase trainer
reward queue p95	verifier 瓶颈	缓存、并行、CPU pool、batch judge
checkpoint pause	I/O 阻塞	async checkpoint / stagger save

8. Reward / Verifier Service：必须是可版本化的在线服务

Reward 不应埋在 trainer 里。平台层最好将其抽象成 `RewardRequest` → `RewardBundle`：输入包含 prompt、response、trajectory、environment state；输出包含每个 reward component、grader version、diagnostics 和不可判定状态。verl 的 `RewardManager` 就允许组合 model-based 与 rule-based reward，并返回最终 token-level reward。[12]

设计点	推荐做法	反模式
版本	grader_version / rule_version	“reward.py 最新版”
分解	correctness / format / safety / cost	只有一个 scalar
缓存	对确定性 verifier 可缓存	重复执行昂贵测试
超时	timeout + abstain	超时按 0 分
可重放	保存 verifier inputs/outputs	无法复现 reward
独立 Eval	训练 reward 与 release eval 分离	训练和验收同一 grader

9. Data & Experience Store：静态数据湖不够

SFT 的主要对象是 dataset；RL 的主要对象是 experience。Experience Store 必须支持流式写入、按 policy_version/task/reward 过滤、TTL、重放、group atomicity 和 lineage。一个 rollout 至少应记录：prompt_id、policy_version、sampling config、token ids/logprobs、reward components、verifier version、group_id、timestamps、environment trace。

Know-how：保留 token ids 和必要 log-prob，比只保存文本更重要。很多 advantage/off-policy/debug 分析无法从纯文本重建。

存储层	典型内容	访问模式
Object Store	datasets / checkpoints / large trajectories	高吞吐顺序读写
Metadata DB	IDs / versions / lineage / status	点查、filter、join
Queue / Stream	fresh rollout / reward jobs	低延迟 producer-consumer
Cache	prefix / verifier / artifacts	热点复用
Analytics Lake	offline failure / cost / slice analysis	批量分析

10. Checkpoint：不是“保存权重”，而是保存可恢复状态

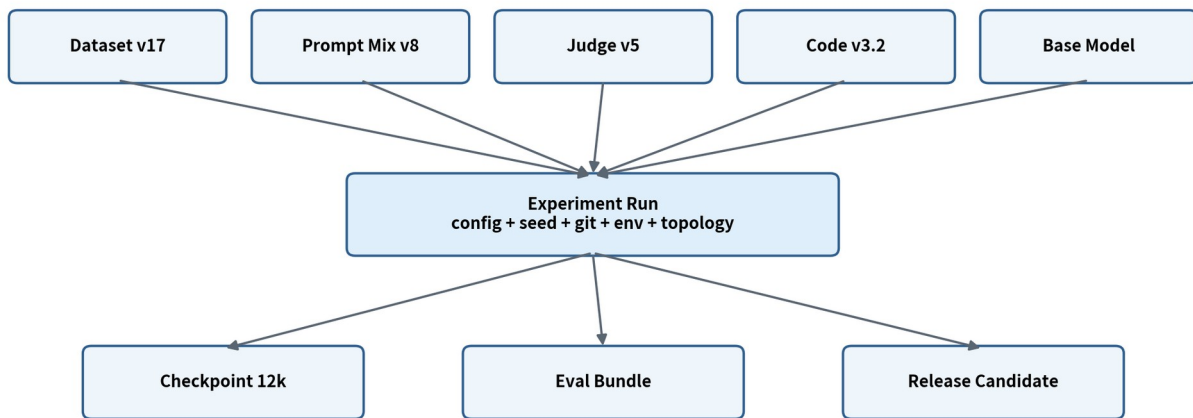
后训练 checkpoint 应区分：模型权重、optimizer state、LR scheduler、RNG state、global step、data cursor、policy/reference version、reward version、parallel topology、experiment config。Megatron Core 的 distributed checkpoint 支持在不同并行配置间加载/reshard，并提供异步 checkpoint 路线；PyTorch DCP 也提供 `async\_save` 以将 checkpoint I/O 从训练关键路径移出。[13][14]

Checkpoint 类型	何时用	权衡
---------------	-----	----

Weights-only	部署 / distill / eval	小、快；不能无损 resume
Full training state	故障恢复	大；I/O 成本高
Reshardable distributed	可能改 TP/PP/DP	灵活；保存/加载更复杂
Async checkpoint	大规模训练	减少 pause；需要后台 I/O/一致性
Milestone checkpoint	关键阶段	长期保存，用于 regression/rollback

工程原则：checkpoint 写入要有 staging → manifest → atomic commit。不要让“目录存在”就等价于 checkpoint 完整。

11. Experiment Tracking 与 Model Registry：重现一次后训练实验需要什么



Reproducibility = Data lineage + Code lineage + Config lineage + Model lineage + Eval lineage

图 4：一次实验必须同时绑定数据、代码、Judge、配置与模型。

MLflow 的 Model Registry 强调 model version、lineage、alias 和 metadata；Dataset Tracking 则将 raw data → model inputs 的数据血缘纳入实验可复现体系。[15][16] 后训练需要比一般 ML 更严格，因为同一 policy checkpoint 在不同 sampling config、reward model 或 eval version 下可产生完全不同结果。

必须记录	示例
Code	git SHA / container digest / dependency lock
Data	dataset version / mixture / filter version
Model	base checkpoint / tokenizer / chat template
Training	optimizer / LR / batch / precision / seed
Parallelism	DP/TP/PP/CP/EP / world size / topology
Rollout	N / temp / top-p / max tokens / engine version

Reward	RM/verifier/rubric version
Eval	benchmark commit / judge / decoding / budget
Hardware	GPU type / interconnect / driver / CUDA

12. Observability: Loss 只是后训练系统的一小部分

需要同时观察 Model Metrics、System Metrics 和 Data Metrics。只看 loss/reward 曲线，会遗漏 rollout 长尾、GPU bubble、queue backlog、reward timeout、checkpoint stall、policy staleness 等系统性问题。

层	核心指标
Model	loss, KL, entropy, reward, pass rate, advantage std
Rollout	samples/s, tokens/s, p50/p95 length, group time, cache hit
Trainer	MFU, step time, grad norm, OOM/retry, collective time
Queue	depth, wait time, drop/TTL, staleness distribution
Reward	latency, timeout, disagreement, invalid/abstain rate
Storage	checkpoint bandwidth, save/load time, object store errors
Cost	GPU-hour / accepted sample, GPU-hour / eval gain

推荐使用统一 `run\_id / trace\_id / prompt\_id / rollout\_id / group\_id / checkpoint\_id` 打通日志和指标，这样一次 eval failure 才能一路追溯到产生它的模型、训练数据和 rollout 配置。

13. Reliability: 大规模后训练最怕 Silent Failure

显式崩溃通常容易发现；真正危险的是 silent corruption：部分 rank 梯度异常、reward 服务版本混用、data cursor 重置导致重复样本、checkpoint 部分写成功、tokenizer/chat template 不一致、异步队列重复消费。

故障	检测	恢复策略
Worker/GPU failure	heartbeat / NCCL timeout	elastic restart / resume checkpoint
OOM	memory telemetry	auto microbatch / retry / quarantine sample
NaN/Inf	grad/loss guard	skip step / rollback / lower LR
Reward service partial failure	error code + abstain	retry / fallback / isolate
Queue duplicate	idempotency key	exactly-once-ish semantic
Partial checkpoint	manifest/commit marker	ignore uncommitted
Version skew	contract validation	fail closed

平台正确性原则：版本不一致、reward 不可判定、checkpoint 不完整时，优先 fail closed，而不是“尽量继续跑”。错误训练信号会被参数永久吸收。

14. Cost Model: 不要用 GPU 数量衡量平台效率

系统优化需要把训练收益和系统成本放在同一张账上。一个 rollout engine 更快，如果生成的是 zero-gradient groups 或重复样本，训练价值可能不升反降。

可采用：

$\text{Useful Learning Efficiency} = \Delta\text{Eval} / (\text{GPU-hours} + \text{RewardCost} + \text{EnvCost} + \text{StorageCost})$

以及 RL 专用：

$\text{Effective Experience Throughput} = \text{Accepted informative groups} / \text{GPU-hour}$

成本	容易被忽略的项
Generation	长尾 completion、N-way sampling、prefix miss
Reward	LLM judge token、unit test、sandbox、external API
Training	reshard、collectives、idle bubble
Storage	trajectory/logprob/checkpoint 体积
Eval	多 seed / judge / environment replay
Failure	重跑、坏 checkpoint、silent corruption 的机会成本

15. 生产级 Post-Training Job Contract

建议将一个 training run 定义成不可变 Contract，而不是散落 CLI。下面是最小结构：

```
run_id: pt-2026-08-13-017
base_model: model-registry://base/v42
data:
  dataset: dataset://posttrain/v118
  mixture: mix://reasoning-agent/v7
rollout:
  engine: vllm
  policy_freshness_max_lag: 2
  samples_per_prompt: 16
reward:
  bundle: reward://rlvr/v9
train:
  algorithm: grpo
  topology: {dp: 8, tp: 4, pp: 2, cp: 2}
checkpoint:
  interval_steps: 100
  async: true
eval:
  suite: eval://release/v31
release_gate:
  require_no_regression: true
```

关键不是 YAML 本身，而是这些 ID 能够被 registry 解引用，并且在半年后仍能完整重建当时的数据、代码、模型、reward、eval 与集群拓扑。

16. 三套典型架构 Recipe

Recipe	架构	适用
A. SFT/DPO Factory	Dataset Registry → FSDP/Megatron Trainer → Eval → Checkpoint Registry	多数中型后训练
B. Synchronous RLVR	Prompt → vLLM Rollout → Verifier → GRPO Trainer → weight refresh	强调 on-policy 正确性
C. Disaggregated RL	独立 rollout/reward/train pools + queue + staleness controller	大规模长 CoT / Agent RL

Recipe B 的关键是 group atomicity、权重刷新和训练/推理 reshard；Recipe C 的关键是 producer-consumer balance、weight broadcast、staleness contract 和异步故障处理。

17. 2025-2026 系统演进：从“RLHF Framework”到“Adaptive Runtime”

系统/工作	关键思想	Know-why
OpenRLHF	Ray + vLLM + DeepSpeed 多模型调度	把 RLHF 多角色拆成可调度资源
HybridFlow / verl	hybrid controller + 3D reshard	同时追求算法表达灵活与执行效率
AReAL	fully async + staleness control	用 freshness 换 overlap
RolloutPipe 2026	complete-group pipelining	在保持 on-policy 下提前训练
DynaResize 2026	runtime GPU role reallocation	静态 rollout/train 切分无法适应长尾
AReAL-Hex	heterogeneous GPU mapping	generation 与 training 的资源特性不同

这条演进线说明：后训练基础设施的前沿问题已经从“怎么分布式跑起来”，转向“如何让资源布局、数据 freshness 和算法语义一起动态适配”。[1][7][9][11][17]

18. 20 类常见 Infrastructure Failure Modes

Failure Mode	症状	修复方向
1. Rollout/Train 固定 50/50	一边长期 idle	队列反馈 + 动态资源
2. 只看 tokens/s	吞吐高但学习慢	看 effective experience
3. 无 policy_version	无法判断 stale	schema 强制版本
4. Reward 无版本	实验不可复现	reward registry
5. Checkpoint 只存 weights	故障无法 resume	full state milestone
6. 保存同步阻塞	step time 周期尖峰	async checkpoint
7. TP 配置训练/推理共用	一边效率差	reshard / separate topology
8. Group 被拆散	GRPO advantage 语义错	group atomic queue
9. 超时 reward 记 0	错误惩罚模型	abstain / retry

10. Queue 无 TTL	训练旧 experience	freshness policy
11. 无 idempotency	重复训练样本	stable IDs
12. Env 不可重放	Agent failure 难调试	record state/action
13. Eval 与 train judge 同源	reward hacking 不可见	independent release eval
14. 数据/模型 registry 分离	血缘断裂	unified lineage
15. 容器/驱动不记录	复现实验失败	environment digest
16. 只做平均 utilization	p95 bubble 被掩盖	stage-level telemetry
17. 不测权重刷新	reshard 成隐藏瓶颈	profile modelflow
18. 节点故障整作业重跑	MTTR 太高	elastic + sharded checkpoint
19. 未设资源配额	一个实验挤占集群	quota / priority
20. 自动继续坏状态	silent corruption	fail closed + rollback

19. Post-Training Platform 成熟度模型

等级	特征	标志
L0 Script	单机/手工文件	跑通一次即可
L1 Reproducible Job	配置化 + checkpoint + basic tracking	可以稳定复现
L2 Distributed Factory	FSDP/TP + registry + rollout service	多实验共享平台
L3 Closed-loop RL Platform	disaggregated roles + reward service + eval gate	持续后训练
L4 Adaptive Runtime	dynamic scheduling + staleness controller + self-healing	按学习效率自动调度

L4 的核心不是更多自动化，而是平台能够用 “queue、freshness、eval gain、failure rate、GPU utilization” 等反馈信号动态改变资源、数据和训练节拍。

20. 90 天落地路线与 Checklist

阶段 A：先建立可复现性（1-30 天）

- 统一 run_id / dataset_id / model_id / judge_id / eval_id；
- 建立 experiment tracking + model/dataset registry；
- full-state checkpoint + resume 演练；
- 建立基础 GPU/step/rollout/reward telemetry。

阶段 B：拆服务和标准 Contract（31-60 天）

- Rollout Engine 独立服务；
- Reward/Verifier Service 独立版本；
- Experience Store + policy_version/group_id；
- SFT/DPO/RL 共用数据和模型 lineage。

阶段 C：做动态控制（61-90 天）

- rollout/train queue feedback;
- staleness cap / TTL / retry;
- async checkpoint;
- release regression gate;
- 按 effective experience/GPU-hour 做 capacity planning。

上线前最后检查：能否回答“这个 checkpoint 用哪批数据、哪个 judge、哪个 rollout config、哪个代码版本、什么 GPU 拓扑训练出来？如果线上退化，能否在小时级回滚并复现？”

21. Know-Why / Know-How 总表

主题	Know-Why	Know-How
多平面架构	RL 是多角色 dataflow	按 Data/Experience/Signal/Train/Control 分层
Rollout	decode 长尾主导节拍	continuous batching + prefix cache + group metrics
Reshard	训练/生成最优拓扑不同	显式 profile weight modelflow
Async	overlap 与 freshness 交易	policy_version + lag cap + queue balance
Scheduler	各 stage 资源曲线不同	feedback-driven role allocation
Checkpoint	故障恢复是正确性需求	full state + atomic commit + async save
Lineage	模型版本本身信息不足	data/code/config/judge/eval 全链路
Observability	loss 看不到系统 bubble	trace + stage metrics + queue metrics
Cost	tokens/s 不是学习收益	Δ Eval / GPU-hour 与 effective group throughput
Release	训练成功不等于可上线	独立 regression suite + rollback

22. Modelflow：权重如何从 Trainer 进入 Rollout

RL 平台除了 dataflow，还存在一条经常被低估的 Modelflow：Trainer 更新参数之后，Rollout Worker 必须看到足够新的 policy。其代价取决于模型大小、并行拓扑、网络带宽、量化/精度格式，以及是全量权重广播还是共享 GPU 上的 reshard。

策略	做法	优点	风险
Full reload	保存 checkpoint 后 rollout 重新加载	语义简单	I/O 与停顿大
Direct broadcast	trainer → rollout 直接分发参数	freshness 好	跨节点网络压力
Shared-memory / co-located	同 GPU/节点切 train-gen 角色	避免远程广播	显存与 reshard 复杂
Incremental/delta	只同步变化或 adapter	带宽小	实现复杂、适用有限
Versioned async refresh	rollout 按版本异步拉取	高 overlap	必须约束 lag

平台应该单独测 `weight\_refresh\_seconds`、`bytes\_per\_refresh`、`refresh\_interval` 和 `policy\_version\_lag`。如果只看 rollout/train step time，权重流可能成为隐藏的第三大瓶颈。HybridFlow 的 3D-HybridEngine 将 training/generation actor reshard 作为核心系统优化，就是这一问题的直接体现。[1]

23. Numerical & Semantic Consistency: Train Engine 与 Inference Engine 必须“讲同一种模型”

训练引擎和推理引擎可能使用不同 kernel、attention implementation、precision、quantization、rope/chat template、stop token，甚至不同 tokenizer build。后训练中这不是普通 serving 差异：rollout 是训练数据来源，因此数值/语义漂移会直接改变优化分布。

一致性对象	测试
Tokenizer	同文本 token ids 完全一致
Chat template	同 messages 序列化完全一致
Special tokens	BOS/EOS/EOT/stop semantics 一致
Logits	固定短输入做 top-k/logit tolerance 对比
Sampling	seed + config 的统计一致性
RoPE/context	长上下文边界测试
Precision	BF16/FP8/quant rollout 的 reward drift A/B
Tool schema	训练/rollout tool serialization 一致

Know-why: RL 的“环境”包含 rollout engine。本质上，推理引擎行为发生变化，就等价于训练环境发生变化。

24. Stage-specific Infrastructure: SFT、DPO、RLVR 不应强行共用同一执行路径

阶段	核心系统瓶颈	最重要的基础设施
SFT	training FLOPs / data loading	FSDP/TP/PP、packing、checkpoint
DPO	chosen/rejected 双序列 forward	pair dataloader、reference logprob cache
RM/PRM	pair/process scoring	balanced sampler、score calibration pipeline
RLHF/PPO	多模型 + rollout + value/reward	role scheduler、reshard、experience store
GRPO/RLVR	N-way long rollout + verifier	parallel sampling、group atomicity、dynamic sampling
Agent RL	environment latency + trajectory	sandbox pool、state replay、tool traces

Distillation	teacher generation + student train	teacher service、verified dataset、cache

统一平台的目标应该是统一 Contract、Registry、Lineage、Observability，而不是把所有训练阶段硬塞进同一个 execution loop。执行引擎可以多样化，但元数据和治理语义应统一。

25. Multi-tenancy 与 Cluster Governance

团队进入持续后训练后，真正稀缺的是集群而不是单次实验。平台需要同时处理交互式小实验、长期 SFT、吞吐型 rollout、release eval 和紧急 regression。没有治理层时，最常见结果是大作业长期占满高端 GPU，而小实验和评测排队，最终降低研发迭代速度。

机制	作用
Quota	限制团队/项目长期占用
Priority Class	release/blocker > research > backfill
Preemption	低优先级可 checkpoint 后让出资源
Gang Scheduling	分布式 job 必须同时获得完整资源
Backfill	利用碎片资源跑 eval/rollout
GPU Affinity	按 NVLink/IB topology 放置 TP/PP
Budget Guard	按 GPU-hour / token / run 设置成本上限

Ray Placement Groups 的 gang scheduling 是 RL 多角色资源编排的一块基础能力。[10] 更高层还需要把 job priority、quota、checkpoint/preemption 和拓扑感知组合起来。

26. Capacity Planning: 如何估算 Rollout / Train 的资源比例

一个实用近似是先测每个 stage 的 service rate，再做 producer-consumer 平衡。假设 rollout 产生有效 experience 的速率为 R_g (groups/GPU-hour)，trainer 消费速率为 R_t (groups/GPU-hour)，分配 G_g 与 G_t 个 GPU；稳态应满足：

$$G_g \times R_g \approx G_t \times R_t$$

但 RL 不能只追求队列稳定，还要加入 freshness 约束：当 backlog 对应的平均等待时间超过允许 policy lag 时，即便 trainer 最终能消费完，也已经违反训练语义。

现象	容量判断
Trainer idle, rollout queue 近 0	rollout 不足
Backlog 持续增长, staleness 上升	trainer 不足或 rollout 过量
两边 utilization 都低	调度/通信/reshard/同步 barrier 问题
平均平衡但 p95 波动大	长尾或 batch/group straggler
Reward queue 独立增长	verifier 是第三瓶颈

因此真正的 capacity planner 至少是三阶段：Rollout → Reward → Train，而 Agent RL 还要加入 Environment capacity。

27. Performance Debugging Playbook：先找 Bubble，再调 Kernel

1. 先画 end-to-end timeline：Rollout / Reward / Reshard / Train / Checkpoint 谁在等谁。
2. 看 GPU utilization 时按角色拆分，不看全作业平均值。
3. 计算 queue depth 与 stage service rate，判断是供给不足还是消费不足。
4. 对 rollout 看 length distribution、group completion time、prefix-cache hit、KV utilization。
5. 对 trainer 看 MFU、collective time、activation memory、optimizer step time。
6. 对 modelflow 测 weight refresh/reshard；对 checkpoint 测同步 pause。
7. 最后才做 kernel/precision/speculative decoding 优化，并用“有效经验/GPU-hour”复验。

常见误区：GPU utilization 低就直接增加 batch。RL 的 idle 往往来自跨 stage barrier、straggler、queue starvation 或权重同步，不一定是 kernel 太小。

28. Open-source Stack 选择矩阵

层	候选	选择依据
Training	PyTorch FSDP2 / Megatron Core / DeepSpeed	模型规模、并行维度、团队熟悉度
RL Orchestration	verl / OpenRLHF / AReaL	算法灵活性、同步/异步、规模
Rollout	vLLM / SGLang	吞吐、sampling、cache、框架集成
Cluster	Ray / Kubernetes / Slurm	actor orchestration、gang scheduling、运维体系
Experiment	MLflow / W&B 等	tracking、registry、团队工作流
Storage	Object store + metadata DB + queue	容量、吞吐、生命周期
Observability	Prometheus/Grafana + traces	统一 metrics/trace/log

选择框架时不要只比较 benchmark throughput。更重要的是：能否表达你的 algorithm/dataflow、能否稳定 checkpoint/resume、是否支持当前模型架构和并行方式、能否得到足够细的 observability，以及升级成本。

参考资料与推荐阅读顺序

[1] HybridFlow: A Flexible and Efficient RLHF Framework (2024) - verl / HybridFlow dataflow、hybrid controller、3D-HybridEngine。 [arXiv:2409.19256](#)

[2] OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework (2024) - Ray + vLLM + DeepSpeed 多角色调度。 [arXiv:2405.11143](#)

- [3] DeepSpeed-Chat (2023) - 大规模 RLHF 训练/推理统一引擎的早期系统化方案。 [arXiv:2308.01320](#)
- [4] NVIDIA Megatron Core Parallelism Strategies Guide - DP/TP/PP/CP/EP 与组合。 [NVIDIA Megatron Core docs](#), accessed 2026-08
- [5] PyTorch FSDP2 fully_shard - per-parameter sharding 与 DeviceMesh。 [PyTorch docs](#), updated 2026-04
- [6] PyTorch DeviceMesh - 多维 process group / topology abstraction。 [PyTorch tutorials](#)
- [7] AReaL: A Large-Scale Asynchronous RL System for Language Reasoning (2025) - fully asynchronous + staleness control。 [arXiv:2505.24298](#)
- [8] vLLM Documentation - continuous batching、prefix caching、parallel sampling、speculative decoding。 [vLLM docs](#), accessed 2026-08
- [9] RolloutPipe (2026) - 保持 on-policy 语义的 complete-group pipelined rollout/train overlap。 [arXiv:2606.26997](#)
- [10] Ray Placement Groups - gang scheduling / PACK / SPREAD。 [Ray docs](#), accessed 2026-08
- [11] DynaResize (2026) - disaggregated post-training 的 runtime GPU reallocation。 [arXiv:2607.22614](#)
- [12] verl HybridFlow Programming Guide - RayPPOTrainer / RewardManager。 [verl docs](#), accessed 2026-08
- [13] Megatron Core Distributed Checkpointing - reshardable / async checkpoint。 [NVIDIA docs](#), accessed 2026-08
- [14] PyTorch Distributed Checkpoint async_save - 异步 checkpointing。 [PyTorch tutorial](#), updated 2026-02
- [15] MLflow Model Registry - model version / alias / lineage。 [MLflow docs](#), accessed 2026-08
- [16] MLflow Dataset Tracking - dataset version / data lineage。 [MLflow docs](#), accessed 2026-08
- [17] AReaL-Hex (2025) - heterogeneous GPU-aware asynchronous RL。 [arXiv:2511.00796](#)

推荐阅读顺序：HybridFlow → OpenRLHF → AReaL → vLLM/Megatron/FSDP2 文档 → RolloutPipe/DynaResize → Checkpoint/Registry/Lineage。先理解“RL dataflow 为什么复杂”，再研究单点性能优化。